2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER) | 979-8-3315-3510-0/25/$31.00 ©2025 IEEE | DOI: 10.1109/SANER64311.2025.00011

Can Large Language Models Discover Metamorphic Relations? A Large-Scale Empirical Study

Jiaming Zhang†, Chang-ai Sun†*, Huai Liu‡, Sijin Dong†
† School of Computer and Communication Engineering, University of Science and Technology Beijing, Beijing, China
‡ Department of Computing Technologies, Swinburne University of Technology, Melbourne, Australia
radon@xs.ustb.edu.cn, casun@ustb.edu.cn, hliu@swin.edu.au, sijin@xs.ustb.edu.cn

***Abstract*—Software testing is a mainstream approach for software quality assurance. One fundamental challenge for testing is that in many practical situations, it is very difficult to verify the correctness of test results given inputs for Software Under Test (SUT), which is known as the oracle problem. Metamorphic Testing (MT) is a software testing technique that can effectively alleviate the oracle problem. The core component of MT is a set of Metamorphic Relations (MRs), which are basically the necessary properties of SUT, represented in the form of relationship among multiple inputs and their corresponding expected outputs. Different methods have been proposed to support the systematic MR identification. However, most of them still rely heavily on test engineers' understanding of the SUT and involve massive manual work.**

Although a few preliminary studies have shown LLMs' viability in generating MRs, there does not exist a thorough and in-depth investigation on their capability in MR identification. We are thus motivated to conduct a comprehensive and large-scale empirical study to systematically evaluate the performance of LLMs in identifying appropriate MRs for a wide variety of software systems. This study makes use of 37 SUTs collected from previous MT studies. Prompts are constructed for two LLMs, *gpt-3.5-turbo-1106* and *gpt-4-1106-preview*, to perform the MR identification for each SUT. The empirical results demonstrate that both LLMs can generate a large amount of MR candidates (MRCs). Among them, 29.86% and 43.79% of all MRCs are identified as the MRs valid for the corresponding SUT, respectively. In addition, 24.59% and 38.63% of all MRCs are MRs that had never been identified in previous studies. Our study not only reinforces LLM-based MR identification as a promising research direction for MT, but also provides some practical guidelines for how to further improve LLMs' performance in generating good MRs.

***Index Terms*—Metamorphic testing, metamorphic relation, large language models, oracle problem, empirical study**

I. Introduction

Software testing is a mainstream approach for software quality assurance and improvement [1]. Given a test input to the Software Under Test (SUT), a systematic mechanism called a test oracle is usually required to verify whether the result of the input is correct. If the result is inconsistent with the expected one, the test input is considered to trigger a fault within the SUT. However, in many practical situations, the test oracle for the SUT is unavailable or requires considerable costs. Therefore, it is challenging to determine the correctness of the execution result, which is called the *oracle problem* [2], [3].

* Chang-ai Sun is the corresponding author.

Several techniques have been proposed to address the oracle problem, including those based on assertions [4] and model checking [5], as well as Metamorphic Testing (MT) [6]–[10]. Among them, MT has received much attention in recent years due to its conceptual simplicity and effectiveness in alleviating the oracle problem. Over the past two decades, MT has gained impressive achievements in detecting many faults for various application scenarios, including search engines [11], [12], autonomous driving systems [13], [14], and machine translation applications [15], [16]. Considerable efforts have also been made to improve the performance MT from different perspectives [17]–[19].

Recently, some techniques have been proposed to identify MRs systematically so as to reduce (*i*) the related costs and (*ii*) the reliance on subjective factors of testers. Typical techniques of systematic MR identification include those based on predefined patterns [20], [21], search-based methods (that is, converting MR identification into the problem of searching parameter values) [22], [23], machine learning based prediction [24], [25], and composition of existing MRs [26], [27]. However, these techniques are also subject to certain limitations: (*i*) Most of them still incur substantial manual costs. For example, testers still need to undertake manual work to provide proper patterns for identifying concrete MRs for specific types of SUT [20], to define fitness function for search-based algorithms [22], or to judge the composability of MRs [27]. (*ii*) Some MR identification techniques, for example, machine learning based methods, are restricted to the SUT in a specific field, e.g., numerical calculation programs. Therefore, it is still a challenging task to improve the effectiveness of MR identification in the general context, which is critical for advancing MT.

Over the past few years, large language models (LLMs) have gradually gained popularity and have demonstrated impressive generation capabilities [28]–[31]. Specifically, LLMs have become increasingly popular for enhancing software testing [32], [33]. It is not surprising to see studies that utilized LLMs to enhance MT [34]–[36]. For instance, Tsigkanos *et al.* [34] devised an LLM-based workflow to extract variables for MT from user manuals of scientific software. Zhang *et al.* [35] reported a case study on using ChatGPT for automatic MR generation in the context of autonomous driving systems (ADSs) testing. According to their experimental results, the proposed technique can effectively generate high-quality MRs,

2640-7574/25/$31.00 ©2025 IEEE
DOI 10.1109/SANER64311.2025.00011

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 14,2026 at 02:04:59 UTC from IEEE Xplore. Restrictions apply.

reducing the manual effort required for MR identification. A recent study [36] also investigated the effectiveness of ChatGPT (*gpt-4*) in generating MRs. The experimental study based on nine SUTs showed that ChatGPT might provide valuable insights for formulating new MR candidates, but human intelligence remains indispensable in rectifying their accuracy and appropriateness. Although these preliminary studies have demonstrated LLMs' viability in MR identification, they did not provide an in-depth and comprehensive evaluation of LLMs' capabilities in the context, particularly due to the following limitations:

- *Scope of experimental objects*. The experimental scale of the above studies is quite limited: Zhang *et al.*'s study [35] only used the parking module of ADS as SUT; although nine SUTs were selected for evaluation in Luu *et al.*'s study [36], they are either basic scientific calculation programs or simple machine learning applications. The limited scope of experimental objects made it challenging to evaluate LLMs' capabilities in MR identification comprehensively.
- *Investigation of prompts*. The prompts designed in these preliminary studies were simple. No evaluation has been conducted to analyze whether these prompts were reasonably designed, and without in-depth investigation on which prompt component would significantly impact the MR identification performance of LLMs.
- *Investigation of MRs*. These preliminary studies were focused on certain aspects of the quality of MRs generated by LLMs (such as correctness and novelty). However, they did not provide comprehensive analysis of LLMs' capabilities on MR identification for various categories of SUT, nor the evaluation and comparison of the MR identification capabilities of different LLMs.

We are thus motivated to conduct a comprehensive and large-scale empirical study for systematically evaluating the LLMs' capabilities in MR identification. In this study, we have collected 37 SUTs from previous MT studies [37], [38] as the experimental objects, including six in C language, 14 in C++, 14 in Java, and three in Python. We have also deliberately designed prompts for MR identification for each SUT. With these prompts, LLMs can produce a list of MR candidates (MRCs), based on which LLMs' capabilities in MR identification could be evaluated. The main contributions of this paper are summarized as follows:

- Two popular LLMs, *gpt-3.5-turbo-1106* and *gpt-4-1106-preview*, have been used to identify MRs for the 37 SUTs, and the MR identification capabilities of these LLMs have been comprehensively evaluated.
- Based on the experimental results, we identify the advantages and limitations of using LLM for MR identification. Further discussed are the implications of LLM-based MR identification and potential research directions.
- The source code of SUTs, prompts, and the scripts used in this study are available in github[1], which can facilitate future research work on LLM-based MR identification.

[1]https://github.com/Radon10043/Empirical-Study-LLM-MR

The remainder of this paper is organized as follows: Section II introduces the background knowledge of MT and LLMs. Section III reports the details of the empirical study, including the research questions, methodology, and experimental settings. Section IV presents an analysis of the experimental results and summarizes the advantages and limitations of LLM-based MR identification. Section V discusses the related studies. Finally, Section VI concludes this paper.

II. Background

A. Metamorphic Testing

The purpose of MT is to detect faults based on the necessary properties of SUT when the test oracle is unavailable or hard to acquire [39]. These necessary properties demonstrate the relations that must be satisfied between multiple inputs and their expected outputs, called MRs. Obviously, a set of MRs is the core component of MT. One representative type of MR can be defined as follows: Let the target function or algorithm be f, input set X is $\{x_1, x_2, \ldots, x_n\}(n \geq 2)$, and the corresponding output set Y is $\{f(x_1), f(x_2), \ldots, f(x_n)\}$. Among them, source input sequence X_s is $\{x_1, x_2, \ldots, x_k\}(1 \leq k < n)$, and corresponding source output set Y_s is $\{f(x_1), f(x_2), \ldots, f(x_k)\}$. Note that source inputs in the X_s can be constructed using any test input generation technique, such as random testing and symbolic execution. Follow-up input set X_f is $\{x_{k+1}, x_{k+2}, \ldots, x_n\}$, which can be constructed based on the source input sequence. The corresponding follow-up output set Y_f is $\{f(x_{k+1}), f(x_{k+2}), \ldots, f(x_n)\}$. It is usually considered that an MR $\mathcal{R}$ is the combination of output relation $\mathcal{R}_o$ and input relation $\mathcal{R}_i$, namely $\mathcal{R} = \langle \mathcal{R}_i(X_s, X_f), \mathcal{R}_o(Y_s, Y_f) \rangle$.

Example. One property of the logarithms function can be presented as $ln(x^2) = 2ln(x)$. This property can be expressed as an MR: Suppose there are two inputs x_1 and x_2, and the relationship between them ($\mathcal{R}_i$) is $x_2 = x_1^2$. The corresponding output of x_1 and x_2 is $ln(x_1)$ and $ln(x_2)$. It is evident that the relationship between outputs ($\mathcal{R}_o$) must satisfy $2ln(x_1) = ln(x_2)$. If the above relationship between $ln(x_1)$ and $ln(x_2)$ is not satisfied, it is considered that the MR triggers a fault in the program that implements the logarithms function.

In general, MT can be implemented through the following five steps:

Step 1: Source inputs are constructed.
Step 2: SUT is executed with source inputs, and corresponding source outputs are gained.
Step 3: Follow-up inputs are constructed from the source input based on input relation $\mathcal{R}_i$.
Step 4: SUT is executed with follow-up inputs, and corresponding follow-up outputs are gained.
Step 5: The outputs of source input and follow-up input are examined according to the output relation $\mathcal{R}_o$. The SUT is considered faulty if the output relation violates a certain MR.

Following the above steps makes it feasible to test the SUT even in the absence of test oracles. MT is a state-of-the-art

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 14,2026 at 02:04:59 UTC from IEEE Xplore. Restrictions apply.

technique recognized by ISO/IEC/IEEE [40] and has been widely used to alleviate oracle problems in various types of SUT, such as [11], [13], [15].

B. Large Language Models

Large Language Models (LLMs) have demonstrated impressive performance in many fields. LLMs are pre-trained using billions of texts available on the Internet, and they can be applied to a wide range of applications, including text generation [41], code completion [42], and program repair [43]. LLM can complete a specific task by providing prompts based on natural language without requiring additional training or data. Consequently, the understanding of LLM for the task and the quality of its output depend heavily on the prompts provided by the user. Creating the appropriate prompt that allows LLM to complete the specified task is called prompt engineering [44]. Existing studies have demonstrated that LLM can achieve good performance in specific tasks when provided with natural language descriptions and a few demonstrations [45].

Depending on the variant of the Transformer [46] architecture, LLMs can be classified into three categories: decoder-only models [47]–[49], encoder-only models [50]–[52], and encoder-decoder models [53]–[55]. In general, decoder-only models can generate output based on natural language descriptions, while encoder-only models and encoder-decoder models can insert the most appropriate token(s). Since MRs are generally identified based on user manuals or documentation rather than constructed by inserting token(s) based on context, decoder-only models are more appropriate for MR identification. Therefore, this paper evaluates the MR identification capability of the two popular decoder-only models, specifically *gpt-3.5* and *gpt-4*.

III. Research Methodology

This section presents the research questions, methodology, and experimental settings.

A. Research Questions

The experimental study is designed to evaluate the MR identification capability of LLMs. More specifically, the study is structured around four main Research Questions (RQs).

RQ1: How does LLM perform in MR identification?

We are interested in whether LLMs are capable of identifying MRs for the SUT. We particularly investigated which MRCs generated by LLM are consistent with the definition of MR, and among such MRCs, which MRCs correctly demonstrate the properties of the SUT.

RQ2: For different categories of SUT, how does LLM perform in terms of MR identification?

We classified SUTs into different categories and evaluated the MR identification capability of LLM for different categories. The classification of SUT was mainly based on two categories: (*i*) Characteristics, i.e., whether the SUT is a numerical calculation program or a non-numerical calculation program; (*ii*) Frequency, i.e., how frequently the SUT was selected in previous MT-related studies. When evaluating the MR identification capability of LLM for different SUT, these two categories have a high reference value.

RQ3: Can LLM generate extra MRs that were not identified previously?

After analyzing the MR identification capability of LLM for various categories of SUT, we studied whether LLMs generate extra MRs (i.e., MRs that have not been identified before). By analyzing extra MRs, we could determine to what extent LLM can improve the MR identification and the whole MT process. On the one hand, these extra MRs may reveal additional properties of the SUT and enhance test adequacy; on the other hand, they are supplementary to existing MRs. The identification of new MRs would not only reduce the manual costs of MT, but also improve its effectiveness.

RQ4: How do different prompt components influence the performance of MR identification of LLM?

An ablation study was conducted whose goal is to determine whether the prompt is designed reasonably and which component significantly impacts the performance of MR identification of LLM.

B. Methodology

The framework of empirical study is shown in Figure 1. There are two main parts in the study: *MR Generation* and *MR Validation*. The following subsection will provide a detailed description of each part.

1) MR Generation: This part is further divided into *Prompts Construction*, *LLM Interaction*, and *MR Candidates Extraction*.

(i) Prompts Construction. To evaluate MR identification capability comprehensively, we need to construct prompts instructing LLMs to perform MR identification. A prompt template is designed in this study to synthesize prompts for MR identification, as shown in Figure 2. The prompt template includes the following three components:

- *Concept Introduction*. The concepts of MT and MR are introduced in prompts, with the goal of helping LLM gain an in-depth understanding of them, resulting in the MRCs generated by LLM being more likely to satisfy the definition of MR.
- *Specification*. The prompts also include the specification of SUT. Upon receiving the prompt, LLM can identify the property of the SUT based on the specification, thereby identifying MRs.
- *Initial Codified MRs*. The output of LLM may be unclear and hard to comprehend if only the text description of MRs is generated. In order to solve this problem, some example MRs are included to construct the few-shot prompt. LLM is expected to generate outputs similar to examples with the existence of *Initial Codified MRs*.

It is worth noting that only a small amount of MRCs can be generated from a single interaction. Generating more MRCs is necessary to comprehensively evaluate the MR identification capability of LLM. Therefore, LLM is repeatedly instructed

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 14,2026 at 02:04:59 UTC from IEEE Xplore. Restrictions apply.

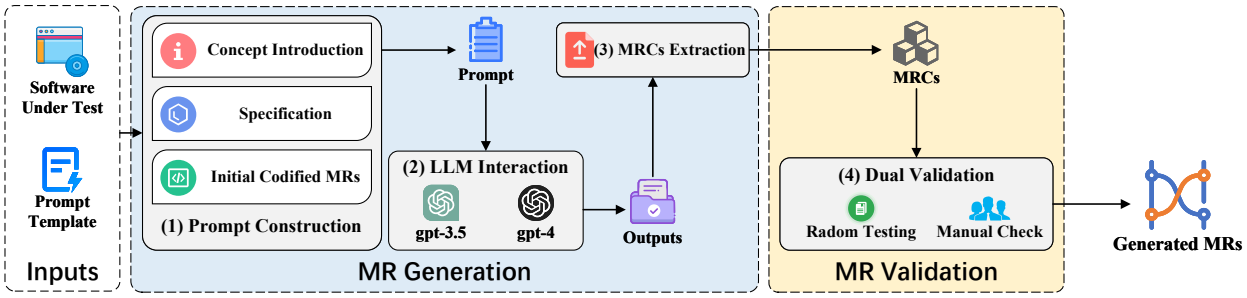


Fig. 1. The framework of the Empirical Study.

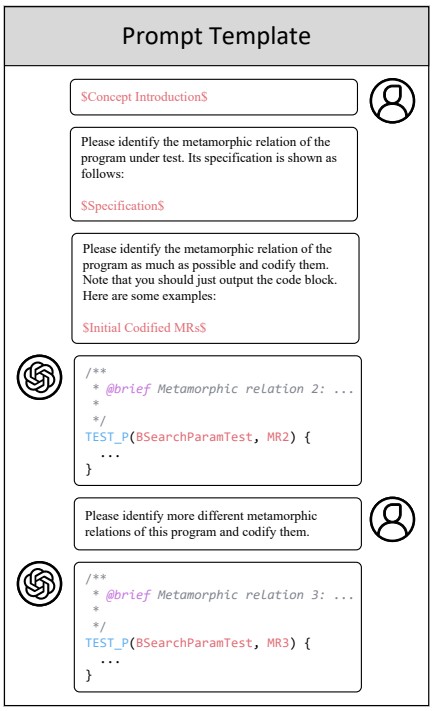


Fig. 2. An Example of Prompt Template

to generate more different MRCs in the subsequent prompts until the termination condition is satisfied. For instance, in Figure 2, "Please identify more different metamorphic relations of this program and codify them." repeatedly prompts LLM to generate more MRCs.

(ii) LLM Interaction. LLM is instructed to perform MR identification based on the prompts constructed in the previous subpart. The interaction process is presented in Algorithm 1, lines 1-9. Its inputs include target LLM llm, termination condition of interaction $cond$ and the prompts constructed in the previous subpart $prompts$. First, the variable $history$ is initialized (lines 2), which records the history of the interaction. Then, a valid prompt is extracted from $prompts$ and saved as $content$. The historical interaction record $history$ may enable LLM to have a higher probability of generating diverse MRCs. Therefore, both $content$ and $history$ are provided as input parameters to instruct LLM to perform MR identification. The MR identification result is stored into $response$, and $history$ is updated according to $content$ and $response$ (lines 4-6). The above process is repeated until the termination condition $cond$ is satisfied, such as the total number of generated MRCs is sufficient, or the token number limitation is reached (lines 3-7). Finally, the interaction is terminated, and $history$ is returned (line 8).

Algorithm 1: The Process of LLM interaction and MRCs extraction

```
Input : Target LLM llm, termination condition cond,
        constructed prompt prompts
Output: MRCs generated by LLM MRCs
1  Function perf_interaction(llm, cond, prompts):
2      history ← empty_list();
3      while cond is not satisfied do
4          content ← prompts.next_valid();
5          response ← llm.interact(content,
             history);
6          history.update(content, response);
7      end
8      return history;
9  end
10 Function extract_candidates(interact_res):
11     MRCs ← empty_list();
12     md_ast ← parse(interact_res);
13     foreach node in md_ast do
14         if node is code block then
15             MRCs ← extract_code(node);
16         end
17     end
18     return MRCs
19 end
20 output ← perf_interaction(llm, cond, prompts);
21 MRCs ← extract_candidates(output);
22 return MRCs;
```

(iii) MRCs Extraction. This subpart includes extracting all MRCs from the interaction results of LLM, as shown in Algorithm 1, lines 10-19. First, the list $MRCs$ that stores all MRCs is initialized to an empty list. The interaction result

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 14,2026 at 02:04:59 UTC from IEEE Xplore. Restrictions apply.

$interact_res$ generated in the previous subpart is treated as markdown text and parsed into an abstract syntax tree. The parsed results are stored in md_ast (lines 11-12). Then, each node in md_ast is traversed. When the type of traversed node is a code block, it means that this node stores the MRC(s) generated by LLM. For such nodes, the content of the code block is extracted and inserted in $MRCs$ (lines 13-17). After all nodes in md_ast are traversed, $MRCs$ is returned (line 18).

2) MR Validation: This stage aims to validate whether MRCs conform to the definition of MR and correctly demonstrate the properties of the SUT. The former concerns the *legality* of an MRC, which is validated manually; the latter concerns the *correctness* of an MRC, which is validated through a *Dual Validation* process which includes (*i*) *Random Testing* and (*ii*) *Manual Checking*.

(i) Random Testing. In order to validate the correctness of MRCs as accurately as possible, a large number of test inputs need to be constructed to cover the input domain of MRCs as much as possible. Manually constructing test inputs alone is unreliable since most corner cases are easily overlooked, and substantial manual costs are incurred. *Random Testing* is a technique that can not only generate large amounts of test inputs but also reduce the manual costs associated with test input construction. Therefore, random testing is used in this study to generate test inputs for each SUT. The execution results of an MRC will help to determine whether an MRC is an MR.

(ii) Manual Checking. Upon the *Random Testing* results, the legality and the correctness of each MRC is also manually validated. The validating process is detailed as follows:

- *Legality validation.* The legality of an MRC is validated through checking whether the MRC satisfies the definition of MR, particularly, whether the MRC contains both input-subrelation and output-subrelation. If one of the subrelation does not exist in the MRC, it is considered as "illegal".
- *Correctness validation.* There are two goals of using *Manual Checking* for correctness validation: On the one hand, some corner cases can be further supplemented manually. With the execution of supplemented test cases, the accuracy of *MR Validation* is further improved. On the other hand, the description and implementation of an MRC may be inconsistent, resulting in false positives or false negatives in *MR Validation*. For such MRC, the implementation of the MRC will be manually fixed based on its description to ensure consistency between description and implementation. *Random Testing* and *Manual Checking* will be reperformed to verify the correctness of the fixed MRC. Accordingly, through the above treatment, false positives or false negatives in the *MR Validation* can be effectively avoided.

After *Manual Checking*, an MRC is added to the list of *Generated MRs* only if it conforms to the definition of MR and correctly illustrates the property of SUT.

```
MRC1: Searching for a value outside the range of the
      array elements should always return false.
MRC2: The result should be the same when searching for
       the same target value in a sub-vector of the
      original vector.
MRC3: If the target is found in the source array,
      adding a new element smaller than all existing
      elements in the array should not affect the
      result.
```

Listing 1. An example of MRs generated by LLMs for binary search function (implementation detail is trimmed)

(iii) Example. Dual Valiadation is illustrated in the following example. Listing 1 shows three MRCs generated by LLM for binary search function. They are denoted as MRC1∼MRC3. First, *Random Testing* is performed with MRC1∼MRC3. Assuming that the implementation of all MRCs is consistent with the corresponding description, MRC1 and MRC3 will pass the *Random Testing*, while MRC2 will not. Next, all MRCs will be validated through *Manual Checking*. Since MRC2 and MRC3 contain input subrelation and output subrelation, while MRC1 does not contain any subrelations. Therefore, MRC1, MRC2, and MRC3 are marked as illegal MRC, legal but incorrect MRC, and correct MRC, respectively. Finally, only MRC3 is added to the list of *Generated MRs*.

C. Experimental Settings

A series of experiments are conducted and the settings are detailed as follows.

1) Experimental Objects: In order to comprehensively evaluate the MR identification capability of LLM, we gathered SUTs from recently published papers [37], [38] as objects. Specifically, a total of 37 objects were gathered in our study, including six C objects, 14 C++ objects, 14 Java objects, and three Python objects. Table I summarizes detailed information about each objects. The column *Language* summarizes the language in which the object is written. The column *LOC* shows the size of the object, that is, the lines of code. Apart from the aforementioned basic information, the characteristics and the frequency of each object were also investigated. Characteristics and frequency are two typical attributes of SUT. They can assist in evaluating the MR identification capability of LLM for different categories of SUT, thus providing references for future LLM-based MR identification studies. The column *Characteristics* shows whether an object is classified into numerical calculation programs (*numerical*) or others (*others*), and it depends on the description and implementation of the object. Moreover, the column *Frequency* specifies how frequently an object was used as an SUT in previous MT-related studies, and its value was calculated as follows:

Step 1: We checked references in [37], [38], and acquired papers that selected these objects and related to MT.

Step 2: We searched *"The name of SUT" AND "metamorphic testing"* in Google Scholar to further acquire papers related to MT and containing the names of objects.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 14,2026 at 02:04:59 UTC from IEEE Xplore. Restrictions apply.

TABLE I
THE INFORMATION OF SOFTWARE UNDER TEST

SUT	Description	Language	Characteristics	Frequency	LOC
ACMS	Aviation Consignment Management System	Java	numerical	Low	171
Boyer	The Boyer-Moore algorithm	Java	other	High	605
Bsearch1	Find the range of a given value	C++	numerical	Low	54
Bsearch2	Perform a binary search on a sorted array	C++	numerical	Low	34
ClosestPair	Find the closest two points in a given set	C++	numerical	Low	169
CUBS	China Unicom Billing System	Java	numerical	Low	232
Determinant1	Calculate the determinant of a square matrix	Java	numerical	Low	99005
DistinctSubsequence	Calculate the number of distinct subsequences	C++	other	Low	19
Editingdistance	Computes the minimum number of operations required to transform one string into another string	C++	other	Low	40
ERS	Expense Reimbursement System	Java	numerical	Low	174
f_oneway	Perform a one-way ANOVA test	Python	numerical	Low	250
FastJson	A Java library that can convert Java Objects into their JSON representation	Java	other	Low	186807
FirstMissingPositive	Finds the smallest positive integer that does not appear in the array	C++	numerical	Low	29
grep	Searching for a pattern in one or more files	C	other	High	94116
HeapSort	Heap sort algorithm for sorting	C++	numerical	Low	60
InterleavingString	Checking whether a given string is an interleaving of two other strings	C++	other	Low	32
JAMA	A Java Matrix Package	Java	numerical	Low	3351
Jodatime	A quality replacement for the Java date and time classes.	Java	other	Low	32739
Klp	Selecting keys to open locks	C++	other	Low	62
Lucene	Text search engine library	Java	other	High	19205
MaxRectangle	Calculate the area of the largest rectangle	Java	other	Low	64
Maxsub	Calculate the maximum subarray sum	Java	other	Low	33
MOS	Meal Ordering System	Java	numerical	Low	217
Multi-MAXSUM	Calculating the maximum sum of multiple contiguous, non-overlapping subarrays	C++	other	Low	43
Printtokens	Lexical analyzer	C	other	High	735
Printtokens2	Lexical analyzer	C	other	High	576
QuickSort	Quick sort algorithm for sorting	C++	numerical	Low	59
rotate	Rotating an array by a specified angle	Python	other	Low	507
RSA	RSA encryption algorithm	C++	numerical	Low	16
schedule	Priority scheduler	C	other	High	417
schedule2	Priority scheduler	C	other	High	379
shortest_path	Computing the shortest paths in a graph	Python	other	Low	1385
Spwiki	Calculating the shortest path distance between two nodes	C++	other	Low	85
Superstring	Computing the shortest superstring from a given set of strings	Java	other	Low	105
SurroundedRegion	Identifying all regions in a matrix that are surrounded	C++	other	Low	94
TCAS	Traffic Collision Avoidance System	C	numerical	High	178
Trisquarej	Evaluating the type of a triangle and calculating the area	Java	numerical	Low	98

Step 3: We merged papers collected from the above two steps and counted the times each object was selected in previous studies.

Step 4: The frequency values were divided into *Low* and *High*. *Low* refers to objects selected as SUT less than five times in previous studies. Otherwise, the object was considered *High* frequency.

2) Metrics: We selected three metrics for this study, including *legal rate* (LR), *correct rate* (CR), and *innovative rate* (IR). LR refers to the proportion of MRCs generated by LLM that conform to the definition of MR out of all generated MRCs. Even though the concept of MR and the expected output format have been introduced to LLM, it may still generates MRCs without input or output subrelations, e.g., MRC1 presented in Listing 1. Such MRCs that lack input or output subrelation are considered as illegal. In other words, an MRC can be regarded as legal only if it contains both input and output subrelations. LR is calculated as Equation (1):

$$LR = \frac{|LMRCs|}{|MRCs|} \times 100\%, \tag{1}$$

where $|LMRCs|$ is the number of legal MRCs, $|MRCs|$ is the number of generated MRCs.

CR refers to the proportion of MRCs that correctly demonstrate the properties of SUT (i.e., MRs) among all MRCs. The correctness of an MRC is checked via the *Dual Validation* process, as mentioned in Section III-B2. During *Dual Validation*, α test cases are generated and executed through *Random Testing*, and such process is repeated β times. Note that for some incomplete MRCs (such as MRCs with incomplete implementation), their correctness is only judged through *Manual Checking*. Considering the time cost, α and β are empirically set to 1000 and 5, respectively. Regarding *Manual Checking*, two postgraduates are assigned to check all MRCs multiple times. If they have different opinions on the correctness of an

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 14,2026 at 02:04:59 UTC from IEEE Xplore. Restrictions apply.

MRC, they would have in-depth discussion until a consensus was reached. Both of them are familiar with MT and thus able to ensure the accuracy of *Manual Checking*. CR is calculated as Equation (2):

$$CR = \frac{|CMRCs|}{|MRCs|} \times 100\%, \tag{2}$$

where $|CMRCs|$ is the number of correct MRCs.

IR is used to measure the proportion of newly discovered MRs among MRCs. Such MRs are called extra MRs in our study. Extra MRs may reveal undiscovered properties of the SUT, improving testing adequacy. Furthermore, they can also serve as a supplement to existing MRs, reducing manual costs while improving effectiveness associated with MR identification. Thus, innovative rate is a vital metric in evaluating the MR identification capability of LLM. In order to filter out extra MRs, MRs used in previous studies are extracted first from the papers collected in Section III-C1. Then, each of the generated MRs is compared with extracted MRs to determine whether it is an extra MR. IR is calculated as Equation (3):

$$IR = \frac{|EMRs|}{|MRCs|} \times 100\%, \tag{3}$$

where $|EMRs|$ is the number of extra MRs.

3) LLMs: Considering the limitations on the number of tokens and the cost, two LLMs, *gpt-3.5-turbo-1106* and *gpt-4-1106-preview*, were leveraged to generate MRCs for SUT. When writing this paper, they were the most advanced LLMs in the *gpt-3.5* and *gpt-4* series, implying the representativeness of this study. These LLMs followed the default settings, i.e., *temperature*=1.0, *top_p*=1.0. For *Initial Codified MRs* in the prompt, most of them continued to use the MRs provided in [38], and ambiguous MRs were replaced. For objects not used in [38], the initial codified MRs were discussed and written. Multiple checks and confirmations had been conducted on all initial codified MRs, making them highly accurate. Additionally, to comprehensively evaluate the MR identification capability of LLM, each LLM was required to generate 40 different MRCs for each SUT.

IV. Results and Analysis

This section presents the empirical results, analysis, and the remarking discussions of this study.

A. MR Identification Effectiveness for Each Object (RQ1)

Figs. 3 and 4 show LR and CR results of MRCs generated by two LLMs for experimental objects, respectively. Note that both of them are displayed in descending order of *gpt-3.5-turbo-1106*.

According to the figures, *gpt-3.5-turbo-1106* and *gpt-4-1106-preview* demonstrated good performance in generating legal MRCs. LR of MRCs generated by *gpt-3.5-turbo-1106* and *gpt-4-1106-preview* ranges between 42.5%∼100% and 62.5%∼100%, respectively. Overall, LR of MRCs generated by *gpt-3.5-turbo-1106* and *gpt-4-1106-preview* are 87.64% and 87.85%. In addition, LR exhibited by the LLM when generating MRCs for a single SUT exceeds 75% in most cases, demonstrating the excellent capability of LLMs in legal MRCs generation.

Although there was no significant difference between the LR of MRCs generated by *gpt-3.5-turbo-1106* and *gpt-4-1106-preview*, the number of legal MRCs generated by *gpt-4-1106-preview* was slightly smaller than that of *gpt-3.5-turbo-1106* (1294 and 1296, respectively). Furthermore, we found that *gpt-4-1106-preview* only generated 33 MRCs for the object *schedule*. For other objects, two LLMs generated 40 MRCs for each. The reason is that the prompt was designed with *gpt-3.5-turbo-1106* characteristics in mind. The interaction with *gpt-4-1106-preview* through the same prompt might lead to different performances, slightly decreasing the number of MRCs and legal MRCs.

In terms of CR's result, *gpt-4-1106-preview* significantly outperformed *gpt-3.5-turbo-1106*. Such a result is expected since the generation of correct MRCs is dependent on logical reasoning and the *gpt-4* series has better reasoning capability than the *gpt-3.5* series, which explains why MRCs generated by *gpt-4-1106-preview* could achieve higher CR than *gpt-3.5-turbo-1106*. Furthermore, a preliminary study we conducted revealed the reasons why LLM failed to identify MRs, such as mistaken output subrelation identification and lack of constraints between source and follow-up inputs. One possible direction for future research is to enhance the quality of prompts or fine-tune LLM. The overall CR of MRCs generated by *gpt-3.5-turbo-1106* and *gpt-4-1106-preview* were 29.86% and 43.79%, respectively. In a word, the empirical results showed that LLM can generate legal MRCs and correct MRCs effectively.

Answer to RQ1: Given the appropriate prompt, LLM can generate legal MRCs and correct MRCs effectively.

B. MR Identification Effectiveness for Different Categories of Objects (RQ2)

Figs. 5 and 6 report the LR and CR of MRCs generated by LLMs based on *Characteristics* and *Frequency* of SUT.

1) Characteristics: According to the Fig. 5, MRCs generated by *gpt-3.5-turbo-1106* and *gpt-4-1106-preview* achieved higher CR for numerical calculation programs and higher LR for non-numerical calculation programs, respectively.

The reason for such results is that MR identification relies on logical reasoning capability, while numerical calculation programs had low requirements for logical reasoning. As a result, LLM may generate more correct MRCs, thus achieve higher CR. However, due to the limited diversity of input subrelations in numerical calculation programs, LLM may only be able to generate a limited number of legal MRCs, which led to the LR is relatively low. Unlike numerical calculation programs, input subrelations for non-numerical calculation programs were diverse, resulting in higher legal rate than those for numerical calculation programs. Specifically, MRCs generated by *gpt-3.5-turbo-1106* achieved an average of LR of 83.59% and CR of 30.16% for 16 numerical calculation

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 14,2026 at 02:04:59 UTC from IEEE Xplore. Restrictions apply.

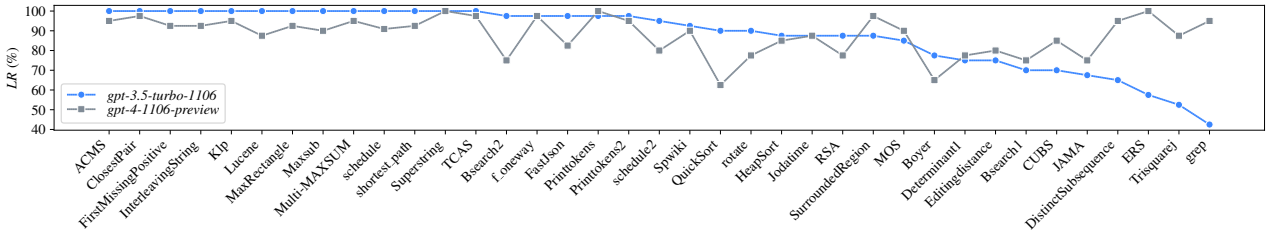


Fig. 3. LR of MRCs generated by *gpt-3.5-turbo-1106* and *gpt-4-1106-preview* for different SUTs

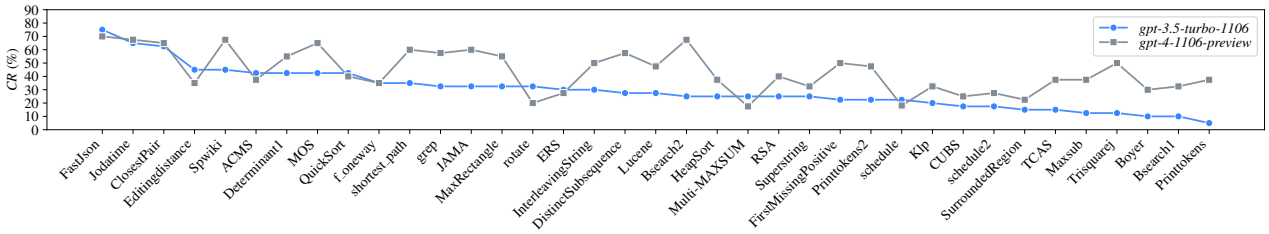


Fig. 4. CR of MRCs generated by *gpt-3.5-turbo-1106* and *gpt-4-1106-preview* for different SUTs

programs, LR of 90.71% and CR of 29.64% for 21 non-numerical programs; MRCs generated by *gpt-4-1106-preview* achieved an average of LR of 85.62% and CR of 45.31% for numerical calculation programs, LR of 89.57% and CR of 42.41% for non-numerical programs.

2) Frequency: Fig. 6 presents the experimental results of LR and CR of MRCs generated by *gpt-3.5-turbo-1106* and *gpt-4-1106-preview* for SUT with various frequencies.

We observed that (*i*) MRCs generated by two LLMs achieved similarly LR for SUT with different frequencies, and (*ii*) MRCs generated by two LLMs achieved high CR for SUT with low frequency. Intuitively, it would be expected that MRCs generated by LLM should achieve higher CR for SUT with high frequency than that of low frequency. One possible explanation for the "counter-intuitive" experimental results is that LLM was trained using a large amount of data collected from the Internet. Consequently, it can effectively generate correct MRCs for the SUTs that are popular on the Internet, such as *ClosestPair* and *Jodatime*, thus MRCs generated by LLMs achieve high CR. On the contrary, the SUTs that are popular in previous MT studies, such as *Printtokens* and *schedule*, are less popular on the Internet; thus, it may be hard for LLM to generate correct MRCs for them effectively. Moreover, these SUTs are non-numerical calculation programs, which further limited the MR identification capability of LLMs, resulting MRCs generated by LLMs with limited CR.

Specifically, MRCs generated by *gpt-3.5-turbo-1106* achieved an average of LR of 87.33% and CR of 32.84% for 29 SUTs with low usage frequency, LR of 88.75% and CR of 19.06% for 8 SUTs with high usage frequency; *gpt-4-1106-preview* achieved an average of LR of 87.59% and CR of 45.26% for SUTs with low usage frequency, LR of 88.86% and CR of 37.90% for SUTs with high usage frequency.

Answer to RQ2: MRCs generated by LLM can achieve higher LR for non-numerical calculation programs than those for numerical programs; however higher CR can be achieved for numerical calculation programs than those for non-numerical programs. Moreover, compared with SUTs with high usage frequency in previous MT studies, LLM was more likely to exhibit impressive MR identification capability for SUTs with low usage frequency.

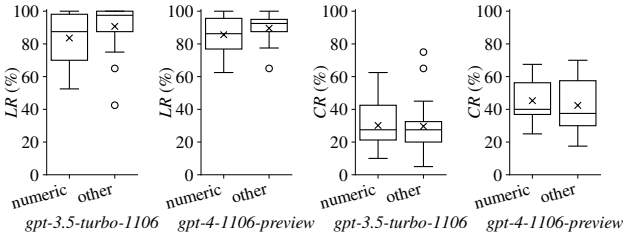


Fig. 5. LR and CR of MRCs Generated by *gpt-3.5-turbo-1106* and *gpt-4-1106-preview* for Different Characteristics of SUT.

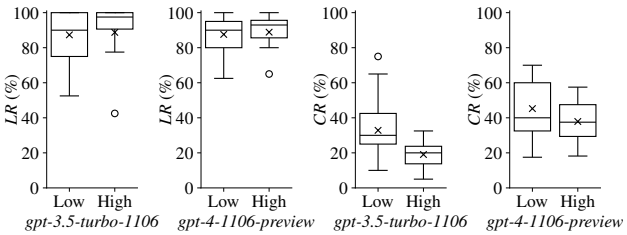


Fig. 6. Experimental Results of LR and CR of MRCs generated by *gpt-3.5-turbo-1106* and *gpt-4-1106-preview* for Different Using Frequency of SUT.

C. Effectiveness of Newly Discovered MRs (RQ3)

IR of MRCs generated by LLMs for each SUT is shown in Fig. 7. The figure illustrates that two LLMs can generate extra MRs effectively. The overall IR of MRCs generated by *gpt-3.5-turbo-1106* and *gpt-4-1106-preview* achieved 24.59% and 38.63%, respectively. As an illustration, two typical extra MRs are as follows:

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 14,2026 at 02:04:59 UTC from IEEE Xplore. Restrictions apply.

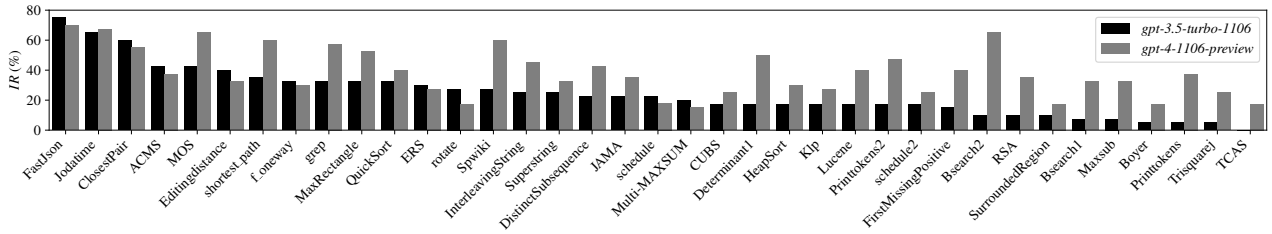


Fig. 7. The Innovative Rate of *gpt-3.5-turbo-1106* and *gpt-4-1106-preview* for each SUT.

MR1. If all points are scaled by a constant factor, the output should be scaled by the same factor. MR1 was generated by *gpt-3.5-turbo-1106* for *ClosestPair*. *ClosestPair* calculates the closest distance between any two coordinates in the input coordinate set. Obviously, after scaling the abscissa and ordinate values of all coordinates by k times (in the corresponding codified MR, the value of k was 2), the shortest distance between coordinates should also be scaled by k times. The SUT can be considered as robust in distance calculation if the shortest distance can still be calculated correctly after scaling coordinates.

MR2. Changing the order of array elements in a JSON array should not affect the deserialized set. MR2 was generated by *gpt-4-1106-preview* for *FastJson*. *FastJson* is a Java library providing various operations on JSON files. It is obvious that permuting the order of elements in a JSON array would result in the same deserialized element set. If permuting the order of elements in a JSON array leads to the deserialized element set being inconsistent with the previous, it implies that *FastJson* has faults in deserializing the JSON array.

Due to the space limitation, it is impossible to extensively discuss all extra MRs in our study, which are provided in the repository[2]. One plausible explanation why LLM can generate extra MRs is that the undiscovered properties of the SUT can be identified from the specification, and extra MRs are generated based on these properties. The other reason is that the undiscovered properties of SUT may be identified by LLM based on massive training data from the Internet, which can also make LLM generate extra MRs.

Answer to RQ3: LLM can effectively generate extra MRs that have not been identified before. Such MRs can supplement existing MRs, reducing manual costs and improving the effectiveness and efficiency of MT.

D. Ablation Study for the Prompt (RQ4)

An ablation study was conducted to validate whether the three components of the prompts were all helpful to the MR identification. First, each component was deleted separately, and the LLM was used to perform MR identification. We use *Prompt*$_{-C}$, *Prompt*$_{-S}$, and *Prompt*$_{-I}$ to represent the prompt whose *Concept Introduction*, *Specification*, and *Initial Codified MRs* were deleted, respectively. Then, MRCs were extracted from the output of LLM, and the LR and CR of MRCs were validated. Considering the costs associated with *Manual Checking*, four objects were randomly selected for the ablation study. The experimental results are shown in Figs. 8 and 9.

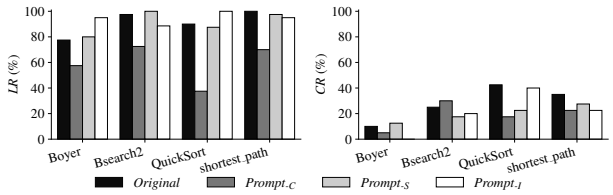


Fig. 8. The Experimental Results for *gpt-3.5-turbo-1106*.

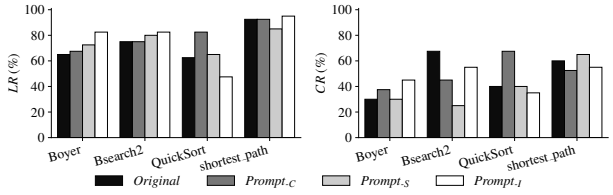


Fig. 9. The Experimental Results for *gpt-4-1106-preview*.

1) gpt-3.5-turbo-1106: It can be seen from the figure that the original LR and CR of MRCs generated by *gpt-3.5-turbo-1106* was better than that of *Prompt*$_{-C}$, *Prompt*$_{-S}$, and *Prompt*$_{-I}$. Such results indicate that all three components were helpful with *gpt-3.5-turbo-1106* in MR identification. In other words, all of them were indispensable.

2) gpt-4-1106-preview: It can be observed that only *Initial Codified MRs* contributed significantly to MR identification. The plausible explanation is that *gpt-4-1106-preview* added training data from diverse domains compared with *gpt-3.5-turbo-1106*. Even if the *Concept Introduction* and *Specifications* were absent, MRCs generated by *gpt-4-1106-preview* can still achieve high LR or CR by incorporating the additional data. However, when *Initial Codified MRs* were absent, the prompt was converted from a few-shot into a zero-shot. Few-shot was more effective than zero-shot for instructing LLM to generate correct MRCs. In a word, *gpt-4-1106-preview* can effectively generate correct MRCs when *Initial Codified MRs* exist.

Answer to RQ4: All three components of the prompt are essential for *gpt-3.5-turbo-1106*. However, for *gpt-4-1106-preview*, only Initial Codified MRs significantly contribute to MR generation.

E. Remarking Discussions

Based on the above empirical results, we further summarize the advantages and limitations of LLM-based MR identifica-

[2]https://github.com/Radon10043/Empirical-Study-LLM-MR

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 14,2026 at 02:04:59 UTC from IEEE Xplore. Restrictions apply.

tion as follows:

Advantages. (*i*) The proposed approach generated many legal but incorrect MRCs that were very close to the MRs. It is possible to convert such MRCs into MRs with minor revision. In other word, these legal but incorrect MRCs can inspire practitioners to infer new MRs effectively and efficiently. (*ii*) Considering the fact that most MR identification techniques generate MRs with limited readability, and converting these into well-readable MRs is labor-intensive, it is of much value to leverage the powerful generation capabilities of LLMs to generate concrete and well-readable MRs directly. (*iii*) Unlike those MR identification techniques that are only applicable to objects in specific domains, such as numerical calculation [23], [25], LLM can generate MRs for any domain as long as appropriate prompts are provided. (*iv*) It is possible for LLMs to generate MRs that have not been identified previously. Such MRs may expose previously unknown faults, thereby contributing to the overall quality assurance of the SUT.

Limitations. (*i*) LLM may generate duplicate or redundant MRs. (*ii*) Manual checking remains indispensable in validating and rectifying the correctness of generated MRCs, which may incur substantial costs when a huge number of MRs are expected to be identified.

V. Related Work

In recent years, significant efforts have been made to improve the effectiveness and efficiency of MR identification from different perspectives [20]–[23], [26], [27], [56]–[64]. The state-of-the-art AI-based MR identification methods can be categorized into machine-learning-based and LLMs-based.

A. Machine learning based MR identification methods

Most existing machine learning based MR identification methods are actually the prediction of MRs, from a set of pre-defined relations, based on features of the target functions or programs.

Kanewala and Bieman [24] proposed an approach for automatically predicting MRs using machine learning techniques. This approach detects potential MRs using features extracted from the control flow graph. In a follow-up study, Kanewala *et al.* [25] developed two graph kernels that extended the extracted features and used them for predicting MRs. However, it is challenging for these methods to obtain the training datasets with labels. To address this challenge, Nair *et al.* [65] proposed a general method to generate a large number of synthetic training data, which may improve the accuracy of MR identification.

On the one hand, machine learning based methods have made significant progress in efficient and effective MR identification. However, on the other hand, such techniques have limited applicability. Most of their applications were restricted to specific domains, such as numerical calculation programs. Different from such methods, LLM-based MR identification methods do not have such limitations in application domains – LLMs are able to identify MRs for any SUT once appropriate prompts have been constructed.

B. LLM-Based MR Identification Methods

Zhang *et al.* [35] conducted a preliminary study on utilizing ChatGPT (*gpt-3.5*) to automatically generate MRs for the parking module in ADSs. According to their experimental results, using ChatGPT for MR identification could be cost-effective and time-saving. However, the quality and relevance of the generated MRs may vary. Luu *et al.* [36] further evaluated the MR identification capability of LLM. Zero-shot prompts were utilized in their study to generate MRs for nine SUT, and the effectiveness of ChatGPT (*gpt-4*) in MR identification was evaluated. Experimental results demonstrated that ChatGPT might provide valuable insights into MR identification. Nevertheless, manual rectification for generated MRs remained indispensable.

Compared with these two preliminary studies, our study is the first to comprehensively evaluate the MR identification capability of LLM from various perspectives through a large-scale empirical study. Furthermore, the advantages and limitations of LLM-based MR identification have been examined, which can serve as a basis for future studies on LLM-based MR identification.

VI. Conclusion and Future Work

In this paper, a comprehensive and large-scale empirical study was conducted to evaluate the MR identification capability of LLM. Based on the empirical results, the main findings of the study can be summarized as follows: (*i*) LLMs were effective in generating legal MRCs and valid MRs. Empirical results showed that the overall LR achieved by LLMs approximately 88% and the overall CR up to 43.79%. (*ii*) LLMs exhibited different MR identification capabilities for different categories of SUT. When SUT was a numerical calculation program or not frequently used in previous studies, LLMs performed impressively in generating MRs. Furthermore, LLMs were more likely to generate legal MRCs for non-numerical calculation programs. (*iii*) LLMs could generate new MRs that have never been reported in previous studies. (*iv*) The optimal prompt varied between different LLMs. Empirical results showed that the prompt designed in this study was reasonable for *gpt-3.5-turbo-1106*, and each prompt component was indispensable for effective MR identification. On the contrary, for *gpt-4-1106-preview*, only the *Initial Codified MRs* had significant contribution.

LLM-based MR identification remains a promising research topic despite some limitations. For future work, we will focus on improving the quality of prompts and optimizing LLM by fine-tuning and hence advancing the performance of MR identification. Another possible direction is to explore decoder-only, encoder-only, and encoder-decoder models for identifying MRs simultaneously, leveraging the advantages of each while compensating for their limitations.

Acknowledgment

This work was supported by the National Natural Science Foundation of China under Grant 62272037 and the Australian Research Council Project under Grant DP210102447.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 14,2026 at 02:04:59 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] J. Tian, *Software quality engineering: testing, quality assurance, and quantifiable improvement*. John Wiley & Sons, 2005.

[2] G. Jahangirova, "Oracle problem in software testing," in *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA'17)*. Association for Computing Machinery, 2017, pp. 444–447. [Online]. Available: https://doi.org/10.1145/3092703.3098235

[3] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, "The oracle problem in software testing: A survey," *IEEE Transactions on Software Engineering*, vol. 41, no. 5, pp. 507–525, 2015.

[4] V. Terragni, G. Jahangirova, P. Tonella, and M. Pezzè, "Evolutionary improvement of assertion oracles," in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE'20)*. Association for Computing Machinery, 2020, pp. 1178–1189. [Online]. Available: https://doi.org/10.1145/3368089.3409758

[5] J. R. Callahan, S. M. Easterbrook, and T. L. Montgomery, "Generating test oracles via model checking," *Technique Report, NASA/WVU Software Research Lab*, 1998.

[6] T. Y. Chen, F.-C. Kuo, H. Liu, P.-L. Poon, D. Towey, T. H. Tse, and Z. Q. Zhou, "Metamorphic testing: A review of challenges and opportunities," *ACM Computing Surveys*, vol. 51, no. 1, jan 2018. [Online]. Available: https://doi.org/10.1145/3143561

[7] W. Wang, J.-t. Huang, W. Wu, J. Zhang, Y. Huang, S. Li, P. He, and M. R. Lyu, "MTTM: Metamorphic testing for textual content moderation software," in *Proceedings of the IEEE/ACM 45th International Conference on Software Engineering (ICSE'23)*, 2023, pp. 2387–2399.

[8] M. Fan, J. Wei, W. Jin, Z. Xu, W. Wei, and T. Liu, "One step further: evaluating interpreters using metamorphic testing," in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA'22)*. Association for Computing Machinery, 2022, pp. 327–339. [Online]. Available: https://doi.org/10.1145/3533767.3534225

[9] M. Zhang, Y. Zhang, L. Zhang, C. Liu, and S. Khurshid, "DeepRoad: GAN-based metamorphic testing and input validation framework for autonomous driving systems," in *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering (ASE'18)*. Association for Computing Machinery, 2018, pp. 132–142. [Online]. Available: https://doi.org/10.1145/3238147.3238187

[10] S. Segura, G. Fraser, A. B. Sanchez, and A. Ruiz-Cortés, "A survey on metamorphic testing," *IEEE Transactions on Software Engineering*, vol. 42, no. 9, pp. 805–824, 2016.

[11] Z. Q. Zhou, S. Xiang, and T. Y. Chen, "Metamorphic testing for software quality assessment: A study of search engines," *IEEE Transactions on Software Engineering*, vol. 42, no. 3, pp. 264–284, 2016.

[12] S. a. o. Alves de Andrade, I. Santos, C. Brito Junior, M. Júnior, S. R.S. de Souza, and M. E. Delamaro, "On applying metamorphic testing: An empirical study on academic search engines," in *Proceedings of the IEEE/ACM 4th International Workshop on Metamorphic Testing (MET'19)*, 2019, pp. 9–16.

[13] Y. Deng, X. Zheng, T. Zhang, H. Liu, G. Lou, M. Kim, and T. Y. Chen, "A declarative metamorphic testing framework for autonomous driving," *IEEE Transactions on Software Engineering*, vol. 49, no. 4, pp. 1964–1982, 2023.

[14] Q.-H. Luu, H. Liu, T. Y. Chen, and H. L. Vu, "A sequential metamorphic testing framework for understanding autonomous vehicle's decisions," *IEEE Transactions on Intelligent Vehicles*, pp. 1–13, 2024.

[15] Z. Sun, J. M. Zhang, M. Harman, M. Papadakis, and L. Zhang, "Automatic testing and improvement of machine translation," in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (ICSE'20)*. Association for Computing Machinery, 2020, pp. 974–985. [Online]. Available: https://doi.org/10.1145/3377811.3380420

[16] D. T. S. Lee, Z. Q. Zhou, and T. H. Tse, "Metamorphic robustness testing of google translate," in *Proceedings of the IEEE/ACM 42nd International Conference on Software Engineering Workshops (ICSEW'20)*. Association for Computing Machinery, 2020, pp. 388–395. [Online]. Available: https://doi.org/10.1145/3387940.3391484

[17] M. Paltenghi and M. Pradel, "MorphQ: Metamorphic testing of the qiskit quantum computing platform," in *Proceedings of the IEEE/ACM 45th International Conference on Software Engineering (ICSE'23)*, 2023, pp. 2413–2424.

[18] M. N. Mansur, V. Wüstholz, and M. Christakis, "Dependency-aware metamorphic testing of datalog engines," in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA'23)*. Association for Computing Machinery, 2023, pp. 236–247. [Online]. Available: https://doi.org/10.1145/3597926.3598052

[19] J. He, Z. Bao, Q. Zhang, W. Sun, J. Liu, C. Fang, and Y. Lin, "Elecdaug: Electromagnetic data augmentation for model repair based on metamorphic relation," in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE'22)*. Association for Computing Machinery, 2023. [Online]. Available: https://doi.org/10.1145/3551349.3559536

[20] S. Segura, J. A. Parejo, J. Troya, and A. Ruiz-Cortés, "Metamorphic testing of RESTful web APIs," in *Proceedings of the 40th International Conference on Software Engineering (ICSE'18)*. Association for Computing Machinery, 2018, p. 882. [Online]. Available: https://doi.org/10.1145/3180155.3182528

[21] T. Y. Chen, P.-L. Poon, and X. Xie, "METRIC: Metamorphic relation identification based on the category-choice framework," *Journal of Systems and Software*, vol. 116, pp. 177–190, 2016. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0164121215001624

[22] J. Zhang, J. Chen, D. Hao, Y. Xiong, B. Xie, L. Zhang, and H. Mei, "Search-based inference of polynomial metamorphic relations," in *Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering (ASE'14)*. Association for Computing Machinery, 2014, pp. 701–712. [Online]. Available: https://doi.org/10.1145/2642937.2642994

[23] B. Zhang, H. Zhang, J. Chen, D. Hao, and P. Moscato, "Automatic discovery and cleansing of numerical metamorphic relations," in *Proceedings of the 35th IEEE International Conference on Software Maintenance and Evolution (ICSME'19)*, 2019, pp. 235–245.

[24] U. Kanewala and J. M. Bieman, "Using machine learning techniques to detect metamorphic relations for programs without test oracles," in *Proceedings of the IEEE 24th International Symposium on Software Reliability Engineering (ISSRE'13)*, 2013, pp. 1–10.

[25] U. Kanewala, J. M. Bieman, and A. Ben-Hur, "Predicting metamorphic relations for testing scientific software: a machine learning approach using graph kernels," *Software Testing, Verification and Reliability*, vol. 26, no. 3, pp. 245–269, 2016. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/stvr.1594

[26] G. Dong, B. Xu, L. Chen, C. Nie, and L. Wang, "Case studies on testing with compositional metamorphic relations," *Journal of Southeast University (English Edition)*, vol. 24, no. 4, pp. 437–443, 2008.

[27] H. Liu, X. Liu, and T. Y. Chen, "A new method for constructing metamorphic relations," in *Proceedings of the 12th International Conference on Quality Software (QSIC'12)*, 2012, pp. 59–68.

[28] R. Meng, M. Mirchev, M. Böhme, and A. Roychoudhury, "Large language model guided protocol fuzzing," in *Proceedings of the 31st Annual Network and Distributed System Security Symposium (NDSS'24)*, 2024.

[29] J. Wang, L. Yu, and X. Luo, "LLMIF: Augmented large language model for fuzzing iot devices," in *Proceedings of the 45th IEEE Symposium on Security and Privacy (SP'24)*. IEEE Computer Society, may 2024, pp. 881–896. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/SP54263.2024.00211

[30] S. Huang, Y. Liu, J. Qi, J. Shang, Z. Xiao, C. Fung, Z. Wu, H. Yang, Z. Luan, and D. Qian, "Gloss: Guiding large language models to answer questions from system logs," in *Proceedings of the 31th IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER'24)*, 2024, pp. 91–101.

[31] G. Morales, P. K. C, S. Jahan, M. B. Hosseini, and R. Slavin, "A large language model approach to code and privacy policy alignment," in *Proceedings of the 31st IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER'24)*, 2024, pp. 79–90.

[32] C. S. Xia, M. Paltenghi, J. Le Tian, M. Pradel, and L. Zhang, "Fuzz4All: Universal fuzzing with large language models," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE'24)*. Association for Computing Machinery, 2024. [Online]. Available: https://doi.org/10.1145/3597503.3639121

[33] M. Schäfer, S. Nadi, A. Eghbali, and F. Tip, "An empirical evaluation of using large language models for automated unit test generation," *IEEE Transactions on Software Engineering*, vol. 50, no. 1, pp. 85–105, 2024.

[34] C. Tsigkanos, P. Rani, S. Müller, and T. Kehrer, "Large language models: The next frontier for variable discovery within metamorphic testing?"

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 14,2026 at 02:04:59 UTC from IEEE Xplore. Restrictions apply.

in *Proceedings of the 30th IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER'23)*, 2023, pp. 678–682.

[35] Y. Zhang, D. Towey, and M. Pike, "Automated metamorphic-relation generation with chatgpt: An experience report," in *Proceedings of the IEEE 47th Annual Computers, Software, and Applications Conference (COMPSAC'23)*, 2023, pp. 1780–1785.

[36] Q.-H. Luu, H. Liu, and T. Y. Chen, "Can ChatGPT advance software testing intelligence? an experience report on metamorphic testing," *arXiv preprint arXiv:2310.19204*, 2023.

[37] C.-A. Sun, H. Dai, H. Liu, and T. Y. Chen, "Feedback-directed metamorphic testing," *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 1, pp. 20:1–20:34, 2023. [Online]. Available: https://doi.org/10.1145/3533314

[38] X. Niu, Y. Sun, H. Wu, G. Li, C. Nie, L. Yu, and X. Wang, "Enhance combinatorial testing with metamorphic relations," *IEEE Transactions on Software Engineering*, vol. 48, no. 12, pp. 5007–5029, 2022.

[39] T. Y. Chen, S. C. Cheung, and S. M. Yiu, "Metamorphic testing: a new approach for generating next test cases," Department of Computer Science, Hong Kong University of Science and Technology, Tech. Rep. Technical Report HKUST-CS98-01, 1998.

[40] "IEEE/ISO/IEC international standard - software and systems engineering–software testing–part 4: Test techniques," *ISO/IEC/IEEE 29119-4:2021(E)*, pp. 1–148, 2021.

[41] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation," in *Advances in Neural Information Processing Systems*, vol. 36. Curran Associates, Inc., 2023, pp. 21 558–21 572. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/file/43e9d647ccd3e4b7b5baab53f0368686-Paper-Conference.pdf

[42] Z. Li, C. Wang, Z. Liu, H. Wang, D. Chen, S. Wang, and C. Gao, "CCTEST: Testing and repairing code completion systems," in *Proceedings of the IEEE/ACM 45th International Conference on Software Engineering (ICSE'23)*, 2023, pp. 1238–1250.

[43] Y. Wei, C. S. Xia, and L. Zhang, "Copiloting the copilots: Fusing large language models with completion engines for automated program repair," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE'23)*. Association for Computing Machinery, 2023, pp. 172–184. [Online]. Available: https://doi.org/10.1145/3611643.3616271

[44] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, and G. Neubig, "Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing," *ACM Computing Surveys*, vol. 55, no. 9, pp. 195:1–195:35, 2023. [Online]. Available: https://doi.org/10.1145/3560815

[45] T. Brown *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, vol. 33. Curran Associates, Inc., 2020, pp. 1877–1901. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfcb4967418bfb8ac142f64a-Paper.pdf

[46] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30. Curran Associates, Inc., 2017, pp. 5998–6008. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf

[47] J. Achiam *et al.*, "GPT-4 Technical Report," *arXiv preprint arXiv:2303.08774*, 2023. [Online]. Available: https://arxiv.org/abs/2303.08774

[48] M. Chen *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2303.08774*, 2021. [Online]. Available: https://arxiv.org/abs/2107.03374

[49] H. Touvron *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023. [Online]. Available: https://arxiv.org/abs/2307.09288

[50] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Association for Computational Linguistics, 2019, pp. 4171–4186. [Online]. Available: https://aclanthology.org/N19-1423

[51] Z. Feng *et al.*, "CodeBERT: A pre-trained model for programming and natural languages," in *Findings of the Association for Computational Linguistics: EMNLP 2020*. Association for Computational Linguistics, 2020, pp. 1536–1547. [Online]. Available: https://aclanthology.org/2020.findings-emnlp.139

[52] D. Guo *et al.*, "GraphCodeBERT: Pre-training code representations with data flow," in *Proceedings of the 9th International Conference on Learning Representations (ICLR'21)*, 2021. [Online]. Available: https://openreview.net/forum?id=jLoC4ez43PZ

[53] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, "CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," in *Proceedings of the 26th Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2021, pp. 8696–8708. [Online]. Available: https://aclanthology.org/2021.emnlp-main.685

[54] W. Ahmad, S. Chakraborty, B. Ray, and K.-W. Chang, "Unified pre-training for program understanding and generation," in *Proceedings of the 26th Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, 2021, pp. 2655–2668. [Online]. Available: https://aclanthology.org/2021.naacl-main.211

[55] Z. Du, Y. Qian, X. Liu, M. Ding, J. Qiu, Z. Yang, and J. Tang, "GLM: General language model pretraining with autoregressive blank infilling," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, S. Muresan, P. Nakov, and A. Villavicencio, Eds. Association for Computational Linguistics, 2022, pp. 320–335. [Online]. Available: https://aclanthology.org/2022.acl-long.26

[56] Z. Q. Zhou, T. H. Tse, F.-C. Kuo, and T. Y. Chen, "Automated functional testing of web search engines in the absence of an oracle," Department of Computer Science, The University of Hong Kong, Tech. Rep. Technical Report TR-2007-06, 2007.

[57] Z. Q. Zhou, L. Sun, T. Y. Chen, and D. Towey, "Metamorphic relations for enhancing system understanding and use," *IEEE Transactions on Software Engineering*, vol. 46, no. 10, pp. 1120–1154, 2020.

[58] C.-A. Sun, A. Fu, P.-L. Poon, X. Xie, H. Liu, and T. Y. Chen, "METRIC+: A metamorphic relation identification technique based on input plus output domains," *IEEE Transactions on Software Engineering*, vol. 47, no. 9, pp. 1764–1785, 2021.

[59] C. Xu, V. Terragni, H. Zhu, J. Wu, and S.-C. Cheung, "MR-Scout: Automated synthesis of metamorphic relations from existing test cases," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 6, pp. 150:1–150:28, 2024. [Online]. Available: https://doi.org/10.1145/3656340

[60] H. Zhu, "JFuzz: A tool for automated java unit testing based on data mutation and metamorphic testing methods," in *Proceedings of the 2nd International Conference on Trustworthy Systems and Their Applications (TSA'15)*, 2015, pp. 8–15.

[61] C.-A. Sun, H. Jin, S. Wu, A. Fu, Z. Wang, and W. K. Chan, "Identifying metamorphic relations: A data mutation directed approach," *Software: Practice and Experience*, vol. 54, no. 3, pp. 394–418, 2024. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/spe.3280

[62] J. Ayerdi, V. Terragni, A. Arrieta, P. Tonella, G. Sagardui, and M. Arratibel, "Generating metamorphic relations for cyber-physical systems with genetic programming: an industrial case study," in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE'21)*. Association for Computing Machinery, 2021, pp. 1264–1274. [Online]. Available: https://doi.org/10.1145/3468264.3473920

[63] A. Blasi, A. Gorla, M. D. Ernst, M. Pezzè, and A. Carzaniga, "MeMo: Automatically identifying metamorphic relations in javadoc comments for test automation," *Journal of Systems and Software*, vol. 181, p. 111041, 2021. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0164121221001382

[64] J. Ayerdi, V. Terragni, G. Jahangirova, A. Arrieta, and P. Tonella, "GenMorph: Automatically generating metamorphic relations via genetic programming," *IEEE Transactions on Software Engineering*, vol. 50, no. 7, pp. 1888–1900, 2024.

[65] A. Nair, K. Meinke, and S. Eldh, "Leveraging mutants for automatic prediction of metamorphic relations using machine learning," in *Proceedings of the 3rd ACM SIGSOFT International Workshop on Machine Learning Techniques for Software Quality Evaluation (MaLTeSQuE'19)*. Association for Computing Machinery, 2019, pp. 1–6. [Online]. Available: https://doi.org/10.1145/3340482.3342741

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 14,2026 at 02:04:59 UTC from IEEE Xplore. Restrictions apply.